

Documentazione del progetto: Blood Segmentation con HemoSet

1. What You Will Do

Il progetto ha come obiettivo la realizzazione di un sistema di **blood segmentation** applicato a immagini e video laparoscopici. In particolare, il sistema deve essere in grado di ricevere in input un frame chirurgico e produrre in output una **maschera di segmentazione del sangue**, cioè una rappresentazione in cui ogni pixel viene classificato come appartenente alla classe **blood** oppure alla classe **background**.

Questo task rientra nella **semantic segmentation**. A differenza della classificazione, dove l'intera immagine riceve una sola etichetta, e a differenza della object detection, dove gli oggetti vengono localizzati tramite bounding boxes, nella semantic segmentation ogni pixel dell'immagine viene classificato. Nel caso di questo progetto, quindi, non basta dire se nell'immagine è presente sangue: bisogna localizzare con precisione quali pixel rappresentano sangue, distinguendoli da tessuti, strumenti chirurgici, riflessi, zone scure e altri elementi della scena operatoria.

Il progetto è legato al tema dell'**hemostasis management**, cioè la gestione e il controllo del sanguinamento durante un intervento chirurgico. Automatizzare almeno in parte questa attività può essere utile perché il sangue può occludere il campo visivo, rendere più difficile l'intervento e richiedere una rapida reazione da parte del chirurgo o degli assistenti. Un sistema automatico di segmentazione può quindi rappresentare un primo passo verso strumenti più avanzati, come sistemi di supporto alla suzione automatica, stima della perdita di sangue o identificazione delle zone critiche di sanguinamento.

1.1 Benchmark con HemoSet

La prima attività del progetto consiste nel lavorare con **HemoSet**, il dataset indicato per il progetto. HemoSet è un dataset specifico per la segmentazione del sangue in ambito chirurgico. È pensato per studiare il problema della blood segmentation in condizioni realistiche, quindi non contiene semplici immagini controllate o artificiali, ma frame acquisiti durante una procedura chirurgica robotica su modello animale.

Il dataset contiene video laparoscopici in cui compaiono scenari di sanguinamento. Le immagini presentano diverse difficoltà tipiche del contesto chirurgico: superfici di tessuto irregolari, illuminazione non uniforme, riflessi lucidi, strumenti chirurgici in movimento, sangue con forme variabili e regioni sanguigne che possono cambiare rapidamente nel tempo. Queste caratteristiche rendono il problema molto più complesso rispetto a una segmentazione basata solo sul colore rosso.

HemoSet include video a 30 FPS, con frame rettificati a risoluzione 640×480. Le annotazioni sono maschere pixel-level con due classi principali: **blood** e **background**. Questo significa che ogni immagine annotata ha una corrispondente maschera in cui i pixel di sangue sono separati dai pixel non appartenenti al sangue. Il progetto deve quindi usare queste immagini e queste maschere per addestrare, validare e testare modelli di segmentazione.

Il benchmark con HemoSet serve a valutare quanto bene un modello riesce a generalizzare in condizioni chirurgiche realistiche. Non è sufficiente ottenere buoni risultati su poche immagini semplici: il modello deve funzionare anche quando il sangue è poco visibile, quando i bordi sono irregolari, quando ci sono riflessi o quando il sangue occupa solo una piccola parte del frame.

1.2 Roadmap operativa del progetto

Di seguito viene descritta una possibile roadmap da seguire per realizzare il progetto in modo ordinato.

Fase 1: Comprensione del task

La prima fase consiste nel capire chiaramente il problema da risolvere. Il task principale è la **blood segmentation**, quindi l'obiettivo è produrre una maschera binaria del sangue. Ogni frame laparoscopico deve essere trasformato in una rappresentazione in cui i pixel di sangue sono separati dal resto della scena.

In questa fase bisogna distinguere bene i diversi task di Computer Vision:

- nella **classification**, il modello assegna una label all'intera immagine;
- nella **object detection**, il modello localizza gli oggetti tramite bounding boxes;
- nella **semantic segmentation**, il modello classifica ogni pixel dell'immagine;
- nella **instance segmentation**, il modello separa anche le diverse istanze degli oggetti.

Per HemoSet, il task principale è semantic segmentation binaria: **blood vs background**.

Fase 2: Analisi del dataset

Dopo aver compreso il task, bisogna analizzare HemoSet. È utile visualizzare alcuni frame e le relative maschere, osservare quanta parte dell'immagine è occupata dal sangue, verificare se il sangue compare in grandi pozze, in piccole regioni, in linee sottili o in zone parzialmente coperte da strumenti.

Questa fase serve anche a identificare i casi difficili. Per esempio, il modello potrebbe confondere sangue e tessuto rossastro, oppure potrebbe scambiare riflessi luminosi per regioni non appartenenti al sangue. Inoltre, alcune macchie o discolorazioni possono sembrare sangue, ma non essere considerate vere regioni da segmentare. Questo è un aspetto importante perché in un'applicazione chirurgica non interessa semplicemente rilevare tutto ciò che è rosso, ma identificare correttamente le aree di sangue rilevanti per l'emostasi.

Durante l'analisi del dataset bisogna quindi osservare:

- numero di video e frame disponibili;
- numero di frame annotati;
- formato delle immagini;
- formato delle maschere;
- distribuzione della classe blood rispetto al background;
- casi di occlusione da strumenti;
- variazioni di illuminazione;
- presenza di riflessi;
- presenza di sangue sottile, fluido o accumulato in pozze.

Fase 3: Preparazione dei dati

La terza fase consiste nella preparazione dei dati per il training. Ogni immagine deve essere letta, trasformata e associata alla propria maschera ground truth. Dal punto di vista computazionale, un'immagine RGB è una matrice di pixel con tre canali: rosso, verde e blu. Per esempio, un frame 640×480 può essere rappresentato come un tensore di dimensione:

480 × 640 × 3

Il modello non vede l'immagine come un essere umano, ma come un insieme di valori numerici. Ogni pixel contiene tre valori, uno per ciascun canale colore. Per poter essere usata da una rete neurale, l'immagine deve essere convertita in tensore, normalizzata e spesso ridimensionata.

Le operazioni principali di pre-processing sono:

- **resize**, per portare tutte le immagini alla stessa dimensione;
- **normalization**, per trasformare i valori dei pixel da `[0, 255]` a un intervallo più stabile, per esempio `[0, 1]`;
- **conversione in tensore**, perché PyTorch lavora con tensori;
- **data augmentation**, per aumentare la variabilità dei dati durante il training.

Nel caso di HemoSet, la data augmentation è molto utile perché le scene chirurgiche sono variabili. Si possono applicare trasformazioni come flip, crop, rotazioni leggere, brightness jitter e variazioni di contrasto. È però fondamentale applicare le trasformazioni in modo coerente: se si ruota l'immagine, bisogna ruotare nello stesso modo anche la maschera, altrimenti immagine e ground truth non saranno più allineate.

Il dataset deve poi essere diviso in:

- **training set**, usato per aggiornare i pesi del modello;
- **validation set**, usato per controllare le prestazioni durante l'addestramento;
- **test set**, usato solo alla fine per valutare il modello finale.

Fase 4: Implementazione della baseline con U-Net o U-Net++

Dopo aver preparato i dati, si parte da una baseline. Una baseline è un primo modello funzionante, semplice ma solido, che serve come riferimento per confrontare approcci più avanzati.

Per questo progetto, la baseline consigliata è **U-Net** o **U-Net++**.

U-Net è una delle architetture più utilizzate nella segmentazione medica perché ha una struttura encoder-decoder molto adatta alla produzione di maschere pixel-level. L'encoder estrae feature dall'immagine e riduce progressivamente la risoluzione spaziale; il decoder ricostruisce progressivamente la maschera alla risoluzione originale. Le skip connections collegano encoder e decoder, permettendo al modello di recuperare dettagli spaziali utili per segmentare correttamente i bordi.

U-Net++ è una versione più avanzata di U-Net. Mantiene la struttura encoder-decoder, ma introduce skip connections più dense e annidate. Questo permette di ridurre il semantic gap tra le feature dell'encoder e quelle del decoder, migliorando potenzialmente la precisione della segmentazione.

In questa fase bisogna quindi:

1. implementare o importare una U-Net o una U-Net++;
2. definire la loss function;
3. addestrare il modello sul training set;
4. validarlo sul validation set;
5. salvare il modello migliore;
6. visualizzare le maschere predette;
7. confrontare le predizioni con le ground truth.

Fase 5: Training e validazione

Durante il training, il modello riceve in input un'immagine laparoscopica e produce una mappa di probabilità. Per ogni pixel, il modello stima la probabilità che quel pixel appartenga alla classe blood.

Il ciclo è il seguente:

```
immagine RGB → pre-processing → tensore → modello → mappa di probabilità →  
maschera predetta
```

Durante l'addestramento, la maschera predetta viene confrontata con la maschera ground truth. La differenza tra predizione e ground truth viene misurata tramite una loss function. Per la segmentazione binaria si possono usare:

- **Binary Cross Entropy**, che valuta l'errore pixel per pixel;
- **Dice Loss**, che misura la sovrapposizione tra maschera predetta e maschera reale;
- combinazioni di BCE e Dice Loss, spesso utili quando la classe positiva occupa pochi pixel.

Nel caso di HemoSet, questa attenzione è importante perché spesso il background occupa la maggior parte dell'immagine, mentre il sangue può essere una regione relativamente piccola. Se non si gestisce bene questo sbilanciamento, il modello potrebbe imparare a predire quasi tutto come background, ottenendo una loss apparentemente buona ma una segmentazione poco utile.

Le metriche principali da usare sono:

- **IoU**, cioè Intersection over Union;
- **Dice coefficient**;
- **precision**;
- **recall**;
- **F1 score**;
- **FPS**, se si valuta la velocità real-time.

Fase 6: Evoluzione verso approcci SOTA

Dopo aver costruito una baseline, il progetto richiede di evolvere verso approcci più moderni. Questo significa provare architetture più avanzate, in particolare modelli basati su Transformer o modelli ottimizzati per la velocità.

Una possibile evoluzione è **Swin-Unet**, che mantiene la struttura a U tipica di U-Net ma sostituisce molte operazioni convoluzionali con blocchi Transformer. Questo permette al modello di catturare meglio relazioni globali tra zone diverse dell'immagine. In una scena chirurgica, infatti, capire se una

regione rossa è sangue può richiedere informazioni di contesto: posizione degli strumenti, tessuto circostante, continuità della regione e forma complessiva dell'accumulo.

Un'altra possibilità è **SegFormer**, un'architettura Transformer-based per semantic segmentation. SegFormer combina un encoder gerarchico con un decoder MLP leggero. Il vantaggio è che produce feature multi-scala e mantiene un decoder relativamente semplice, ottenendo un buon compromesso tra accuratezza ed efficienza.

Infine, si può valutare anche una soluzione **YOLO-based segmentation**, soprattutto se l'obiettivo è raggiungere prestazioni real-time. YOLO è nato per object detection, ma le versioni moderne supportano anche la segmentation. Questo approccio può essere utile quando si vuole una pipeline veloce, capace di elaborare frame chirurgici a 30 FPS o più.

Fase 7: Dual-Stage Localization

Una parte più avanzata del progetto è la **dual-stage localization**, cioè una pipeline a due stadi.

Nel primo stadio, il sistema deve rispondere alla domanda:

Dove si trova il sangue?

Qui entra in gioco la segmentation. Il modello produce una maschera che identifica le regioni di sangue nel frame.

Nel secondo stadio, il sistema deve provare a rispondere alla domanda:

Da dove proviene il sangue?

Questo secondo problema è più complesso perché richiede di considerare il tempo. In un singolo frame si può vedere dove si trova il sangue, ma per capire da quale zona proviene bisogna osservare come la regione sanguigna evolve nei frame successivi.

Una possibile pipeline è:

```
video laparoscopico → estrazione dei frame → segmentazione del sangue frame per frame → confronto temporale delle maschere → stima della direzione di espansione → identificazione della possibile sorgente
```

Per implementare questa parte si possono usare strategie diverse. Una soluzione semplice consiste nel confrontare maschere consecutive e osservare dove compaiono nuovi pixel classificati come sangue. Le nuove regioni possono indicare la direzione da cui il sangue si sta espandendo. Una soluzione più avanzata può usare optical flow o tracking, in modo da seguire il movimento del sangue e delle regioni segmentate nel tempo.

Il risultato finale della dual-stage localization non è solo una maschera del sangue, ma una rappresentazione più utile per la chirurgia assistita: una stima della regione sanguigna e una possibile indicazione della zona da cui il sanguinamento ha origine.

Fase 8: Ottimizzazione real-time

Il progetto richiede anche di considerare il vincolo real-time. I video laparoscopici devono poter essere processati a frame rate chirurgico, cioè almeno **30 FPS**. Questo significa che il modello deve elaborare ogni frame in circa 33 millisecondi o meno.

Per valutare questa parte, bisogna misurare:

- tempo medio di inferenza per frame;
- FPS effettivi;
- uso della GPU;
- memoria richiesta;
- stabilità della predizione su video.

Se il modello è troppo lento, si possono usare strategie di ottimizzazione:

- riduzione della risoluzione di input;
- uso di backbone più leggeri;
- mixed precision;
- esportazione del modello in ONNX;
- ottimizzazione con TensorRT;
- quantizzazione;
- fusione di layer;
- pruning o scelta di modelli più piccoli.

Questa fase è importante perché un modello molto accurato ma troppo lento non sarebbe realmente utilizzabile in un contesto chirurgico. Allo stesso tempo, un modello molto veloce ma poco accurato potrebbe produrre maschere inaffidabili. Il progetto deve quindi cercare un compromesso tra accuratezza e velocità.

2. Suggested Tools & Resources

2.1 Dataset: HemoSet

Il dataset principale da usare è **HemoSet**, cioè il primo dataset specializzato per blood segmentation finalizzato all'automazione dell'emostasis management. HemoSet contiene video chirurgici in cui il sangue compare in condizioni realistiche, con geometrie irregolari e scene complesse.

Il dataset contiene immagini laparoscopiche e maschere di segmentazione. Ogni maschera identifica quali pixel appartengono al sangue e quali invece appartengono al background. Questo lo rende adatto a un task di semantic segmentation binaria.

La caratteristica importante di HemoSet è che non si limita a casi semplici o artificiali. Le immagini sono state raccolte in un contesto chirurgico robotico, con condizioni difficili come tessuti eterogenei, luce riflettente, movimento degli strumenti e sangue che si accumula in forme non regolari. Questo rende il dataset molto utile per addestrare modelli che devono funzionare in scenari realistici.

Nel progetto, HemoSet deve essere usato per:

- addestrare i modelli di segmentation;
- validare le prestazioni durante il training;
- testare il modello finale;
- confrontare diverse architetture;
- valutare la robustezza in presenza di casi difficili;
- misurare la velocità su sequenze video.

Un aspetto importante è che il sangue non va trattato semplicemente come “colore rosso”. Alcuni tessuti possono avere tonalità simili, alcune macchie possono non essere regioni sanguigne rilevanti e alcuni riflessi possono alterare l'aspetto del frame. Il modello deve quindi imparare feature più complesse, combinando colore, forma, contesto e posizione.

2.2 Framework: PyTorch

PyTorch è il framework principale per implementare e addestrare i modelli di deep learning. Nel progetto può essere usato per costruire la rete neurale, definire la loss function, gestire il training loop e sfruttare la GPU.

PyTorch lavora con tensori, cioè strutture dati multidimensionali. Nel caso del progetto, un'immagine laparoscopica RGB viene trasformata in un tensore, per esempio con dimensione:

```
batch_size × canali × altezza × larghezza
```

Quindi un batch di immagini RGB può avere forma:

```
B × 3 × H × W
```

dove **B** è il numero di immagini nel batch, **3** sono i canali RGB, **H** è l'altezza e **W** è la larghezza.

Il ciclo tipico in PyTorch è:

1. caricare immagine e maschera;
2. applicare trasformazioni;
3. passare l'immagine al modello;
4. ottenere la predizione;
5. confrontare la predizione con la ground truth;
6. calcolare la loss;
7. fare backpropagation;
8. aggiornare i pesi del modello;
9. ripetere il processo per molte epoche.

PyTorch è particolarmente utile perché permette molta flessibilità. Si può implementare una U-Net da zero, usare librerie esterne, modificare il decoder, cambiare loss function, aggiungere metriche e sperimentare facilmente con architetture diverse.

2.3 Framework: Torchvision

Torchvision è una libreria collegata a PyTorch e contiene strumenti specifici per la Computer Vision. Nel progetto è utile soprattutto per il pre-processing e la data augmentation.

Una cosa importante è che, in segmentazione, le trasformazioni devono essere applicate sia all'immagine sia alla maschera. Per esempio, se si fa un crop dell'immagine, bisogna applicare lo stesso crop anche alla maschera. Se si fa un flip orizzontale del frame, bisogna fare lo stesso flip anche sulla ground truth.

Torchvision può essere usato per trasformazioni come:

- resize;
- crop;
- random horizontal flip;
- conversione in tensore;
- normalizzazione;
- trasformazioni su immagini, video, maschere e bounding boxes.

Nel progetto HemoSet, Torchvision può quindi aiutare a costruire una pipeline dati corretta, evitando errori di disallineamento tra immagine e maschera.

2.4 Framework: MONAI

MONAI, cioè Medical Open Network for AI, è un framework pensato per il medical imaging. È costruito sopra PyTorch e fornisce strumenti specifici per applicazioni mediche, tra cui data loading, trasformazioni, loss, metriche e architetture.

MONAI è utile in questo progetto perché HemoSet non è un dataset generico, ma un dataset medico-chirurgico. La segmentazione del sangue in laparoscopia presenta problemi diversi rispetto alla segmentazione di oggetti comuni: le regioni possono avere forme irregolari, bordi difficili, basso contrasto e forte sbilanciamento tra classe positiva e background.

MONAI può essere usato per:

- gestire trasformazioni mediche;
- implementare pipeline di training più strutturate;
- usare loss adatte alla segmentazione;
- calcolare metriche come Dice;
- usare modelli già implementati o componenti utili per la segmentazione.

In pratica, PyTorch può essere considerato il framework base, Torchvision lo strumento per la Computer Vision generale, mentre MONAI aggiunge componenti specifici per il medical imaging.

3. Architetture di segmentazione

3.1 Ciclo generale: dall'immagine in input alla maschera finale

Indipendentemente dall'architettura scelta, il ciclo generale di un sistema di blood segmentation è simile.

Il punto di partenza è un frame laparoscopico RGB. L'immagine viene acquisita dalla camera endoscopica e rappresentata come una matrice di pixel. Ogni pixel contiene tre valori, corrispondenti ai canali rosso, verde e blu.

Il ciclo generale è:

immagine RGB → pre-processing → tensore → modello di segmentazione → mappa di probabilità → soglia → maschera binaria

Durante il pre-processing, l'immagine viene ridimensionata, normalizzata e convertita in tensore. Successivamente entra nel modello, che estrae feature visive utili. Queste feature possono essere locali, come bordi, texture e cambiamenti di colore, oppure globali, come la posizione della regione sanguigna nella scena e il contesto chirurgico.

Il modello produce una mappa di probabilità. In una segmentazione binaria, ogni pixel riceve un valore tra 0 e 1:

$P(\text{blood} \mid \text{pixel})$

Un valore vicino a 1 indica che il pixel probabilmente appartiene al sangue. Un valore vicino a 0 indica che probabilmente appartiene al background. Applicando una soglia, per esempio 0.5, si ottiene la maschera finale:

- se $p > 0.5$, il pixel viene classificato come blood;
- se $p \leq 0.5$, il pixel viene classificato come background.

La maschera finale è quindi un'immagine binaria in cui il sangue viene evidenziato rispetto al resto della scena.

3.2 U-Net

U-Net è una delle architetture più usate per la segmentazione medica. È adatta a questo progetto perché produce una maschera pixel-level e conserva una buona precisione spaziale.

La struttura di U-Net è composta da due parti principali:

1. **encoder**, detto anche contracting path;
2. **decoder**, detto anche expanding path.

Encoder

L'encoder riceve l'immagine in input e la elabora attraverso layer convoluzionali. Le convoluzioni servono a estrarre feature locali dall'immagine, come bordi, texture, zone rosse, variazioni di intensità e pattern visivi.

All'inizio della rete, le feature sono semplici. Il modello può rilevare bordi, cambiamenti di colore e piccole texture. Andando più in profondità, le feature diventano più astratte: il modello inizia a distinguere regioni compatibili con sangue, tessuto, strumenti chirurgici o riflessi.

Durante l'encoder, la risoluzione spaziale viene progressivamente ridotta. Per esempio, un'immagine può passare da:

480 × 640

a:

240 × 320

poi a:

120 × 160

poi a:

60 × 80

Questa riduzione permette al modello di vedere un contesto più ampio. Se la rete guardasse solo pixel isolati o piccoli gruppi di pixel, potrebbe confondere una regione rossa del tessuto con sangue. Riducendo la risoluzione e aumentando il numero di feature maps, l'encoder costruisce una rappresentazione più semantica della scena.

Quindi, nell'encoder accadono due cose:

- la dimensione spaziale diminuisce;
- il numero di canali o feature maps aumenta.

L'immagine diventa più piccola, ma più ricca di informazione.

Bottleneck

Alla fine dell'encoder si trova il **bottleneck**, cioè il punto più profondo della rete. Qui l'immagine è stata compressa in una rappresentazione compatta. Il bottleneck non conserva tutti i dettagli pixel-level, ma contiene informazioni ad alto livello sulla scena.

Nel caso della blood segmentation, il bottleneck dovrebbe contenere informazioni come la presenza di sangue, la relazione tra sangue e tessuto circostante, il contesto chirurgico e le zone potenzialmente rilevanti per l'emostasi.

Decoder

Il decoder svolge il compito opposto dell'encoder. A partire dalla rappresentazione compressa del bottleneck, ricostruisce progressivamente una mappa alla risoluzione originale.

Questa fase usa operazioni di upsampling o transpose convolution. Per esempio:

60 × 80

diventa:

120 × 160

poi:

240 × 320

poi:

480 × 640

Alla fine, il decoder produce una mappa con la stessa dimensione dell'immagine di input. Questa mappa contiene, per ogni pixel, uno score o logit che indica quanto il modello ritiene probabile che quel pixel appartenga al sangue.

Skip connections

Il problema dell'encoder è che, riducendo la risoluzione, può perdere dettagli spaziali. Questi dettagli sono però fondamentali in segmentazione, perché bisogna sapere con precisione dove iniziano e finiscono le regioni di sangue.

Per questo U-Net usa le **skip connections**. Le skip connections collegano i livelli dell'encoder con i livelli corrispondenti del decoder. In questo modo, il decoder non ricostruisce la maschera solo a partire dal bottleneck, ma riceve anche informazioni ad alta risoluzione provenienti dall'encoder.

Questo è molto importante per HemoSet, perché il sangue può avere bordi irregolari, regioni sottili e piccole macchie. Senza skip connections, il modello potrebbe riconoscere la zona generale del sangue ma produrre una maschera troppo grossolana. Con le skip connections, invece, il modello combina informazione semantica profonda e informazione spaziale fine.

Output di U-Net

Alla fine della rete, U-Net produce una mappa di probabilità. Nel caso binario, si usa spesso una funzione sigmoid, che trasforma ogni valore in una probabilità tra 0 e 1.

Il risultato finale è una maschera binaria:

immagine RGB → encoder → bottleneck → decoder + skip connections → mappa di probabilità → binary mask

Per questo motivo U-Net è una baseline molto solida per la segmentazione medica.

3.3 U-Net++

U-Net++ è un'evoluzione di U-Net. Mantiene l'idea encoder-decoder, ma modifica il modo in cui encoder e decoder comunicano.

In U-Net classica, ogni livello dell'encoder viene collegato direttamente al livello corrispondente del decoder tramite una skip connection. Questo è utile, ma può creare un problema: le feature dell'encoder e quelle del decoder possono avere livelli semantici molto diversi.

Le feature iniziali dell'encoder sono molto locali: bordi, colori, texture. Le feature del decoder, invece, sono più vicine alla decisione finale di segmentazione. Questa differenza viene chiamata **semantic gap**.

U-Net++ cerca di ridurre questo semantic gap usando skip connections dense e annidate. Invece di collegare direttamente encoder e decoder con un unico passaggio, introduce blocchi intermedi che raffinanano progressivamente le feature.

Il ciclo può essere descritto così:

```
immagine RGB → encoder → feature multi-livello → skip pathways annidati →  
decoder → probabilità pixel-level → maschera
```

Nel progetto HemoSet, U-Net++ può essere utile perché la distinzione tra sangue e background non dipende solo dal colore rosso. Alcune zone dei tessuti possono avere colore simile al sangue, e alcuni riflessi possono alterare la percezione della scena. U-Net++ può aiutare perché combina meglio feature locali e feature semantiche, producendo maschere potenzialmente più precise.

3.4 Swin-Unet

Swin-Unet è un'architettura più moderna che mantiene la struttura a U, ma usa componenti basate su Transformer.

Le CNN, come quelle usate in U-Net, sono molto efficaci nel catturare informazioni locali. Analizzano piccoli intorni di pixel e sono quindi adatte a riconoscere bordi, texture e dettagli. Tuttavia, le convoluzioni standard possono avere difficoltà nel modellare relazioni a lungo raggio tra zone lontane dell'immagine.

In una scena chirurgica, il contesto globale può essere molto importante. Per capire se una regione rossa è sangue, il modello potrebbe dover considerare anche la posizione degli strumenti, il tessuto circostante, la forma della regione e la continuità con altre aree del frame.

I Transformer usano meccanismi di **attention**. L'attention permette al modello di capire quali parti dell'immagine sono più rilevanti rispetto ad altre. In questo modo, il modello non si limita a guardare un piccolo vicinato, ma può imparare relazioni più ampie tra regioni diverse.

In Swin-Unet, l'immagine viene divisa in piccole patch. Ogni patch viene trasformata in un vettore, chiamato token. I token vengono poi elaborati da blocchi Swin Transformer.

Il ciclo iniziale è:

```
immagine RGB → divisione in patch → patch embedding → sequenza di token
```

Swin Transformer non calcola l'attention su tutta l'immagine in modo diretto, perché sarebbe troppo costoso. Usa invece finestre locali e finestre spostate, chiamate **shifted windows**. In questo modo riesce a catturare relazioni locali e a far comunicare regioni vicine mantenendo un costo computazionale più controllato.

Come U-Net, anche Swin-Unet ha encoder, bottleneck, decoder e skip connections:

```
immagine RGB → patch embedding → Swin Transformer encoder → bottleneck → Swin Transformer decoder → skip connections → output pixel-level → maschera
```

Nel progetto, Swin-Unet può essere usato come evoluzione della baseline. U-Net è più semplice e spesso più immediata da implementare, mentre Swin-Unet può essere più potente nel catturare relazioni globali, ma può richiedere più risorse computazionali.

3.5 SegFormer

SegFormer è un'architettura Transformer-based per semantic segmentation. È interessante perché cerca di combinare accuratezza ed efficienza.

La sua struttura è composta da due parti principali:

1. un **encoder Transformer gerarchico**;
2. un **decoder MLP leggero**.

L'encoder gerarchico produce feature a diverse scale. Questo è importante perché il sangue può comparire in forme molto diverse: grandi pozze, regioni sottili, piccole macchie o aree parzialmente occluse. Un modello multi-scala può analizzare sia dettagli locali sia contesto globale.

L'encoder produce quindi:

- feature ad alta risoluzione, utili per bordi e dettagli;
- feature intermedie, utili per strutture più ampie;
- feature a bassa risoluzione, utili per il contesto globale.

Il decoder di SegFormer è più leggero rispetto ad altri decoder complessi. È basato su MLP e aggrega le feature multi-scala prodotte dall'encoder. Questo permette di ottenere una mappa di segmentazione senza rendere il modello troppo pesante.

Il ciclo generale è:

```
immagine RGB → encoder Transformer multi-scala → feature gerarchiche → decoder MLP → logits pixel-level → maschera finale
```

Nel progetto, SegFormer può essere usato per confrontare una baseline CNN-based, come U-Net, con un approccio Transformer-based più moderno. La valutazione deve considerare sia la qualità della segmentazione sia la velocità di inferenza.

3.6 YOLO-based segmentation

YOLO-based segmentation è un approccio diverso rispetto a U-Net, U-Net++ e SegFormer. YOLO nasce principalmente per la object detection, cioè per individuare oggetti tramite bounding boxes, ma le versioni moderne supportano anche task di segmentation.

Nel caso della segmentation, YOLO non produce solo un rettangolo attorno all'oggetto, ma anche una maschera associata. Questo può essere utile quando si vuole una pipeline veloce e adatta al real-time.

Il ciclo generale è:

```
immagine RGB → backbone CNN → feature maps → detection head + segmentation head  
→ boxes + classi + maschere
```

Il **backbone** estrae feature dall'immagine. Queste feature vengono poi usate da diverse teste del modello. La detection head predice la posizione degli oggetti, mentre la segmentation head produce le maschere.

La differenza principale rispetto a U-Net è che U-Net produce una mappa completa dell'immagine, classificando ogni pixel. YOLO segmentation è spesso più orientato alla instance segmentation, cioè alla segmentazione di oggetti individuati.

Nel caso di HemoSet, questa distinzione è importante. Se il sangue viene trattato come una regione continua da segmentare, U-Net o SegFormer sono scelte più naturali. Se invece si vuole una soluzione molto veloce che individua e segmenta regioni di sangue come oggetti o istanze, una soluzione YOLO-based può essere interessante.

Il vantaggio principale di YOLO è la velocità. Per questo motivo può essere molto utile nella parte real-time del progetto, dove l'obiettivo è elaborare video laparoscopici ad almeno 30 FPS.

4. Output, loss e valutazione

Indipendentemente dall'architettura scelta, il modello produce una predizione pixel-level. Nel caso binario, l'output è una mappa di probabilità:

```
P(blood | pixel)
```

Durante il training, questa mappa viene confrontata con la maschera ground truth. La ground truth rappresenta la segmentazione corretta del sangue.

La loss function misura la differenza tra predizione e ground truth. Le loss più utili sono:

- **Binary Cross Entropy**, utile per la classificazione pixel-wise;
- **Dice Loss**, utile per misurare la sovrapposizione tra maschera predetta e ground truth;
- **BCE + Dice Loss**, spesso efficace nei casi di class imbalance.

Per la valutazione finale si possono usare:

- **IoU**, che misura il rapporto tra intersezione e unione delle maschere;
- **Dice coefficient**, che misura la sovrapposizione tra predizione e ground truth;
- **precision**, che misura quanti pixel predetti come sangue sono davvero sangue;
- **recall**, che misura quanti pixel di sangue reali sono stati trovati;
- **F1 score**, che combina precision e recall;
- **FPS**, per valutare la velocità del modello su video.

Queste metriche vanno interpretate insieme. Un modello con alta precision ma bassa recall potrebbe segmentare solo le regioni più evidenti di sangue, perdendo molte aree reali. Un modello con alta recall ma bassa precision potrebbe invece classificare come sangue troppe zone non corrette. Nel contesto chirurgico, entrambi gli errori sono importanti.

5. Real-time optimization con TensorRT

Una parte importante del progetto riguarda l'ottimizzazione per il real-time. Il modello deve poter elaborare video laparoscopici a frame rate chirurgico, cioè almeno 30 FPS.

Dopo aver addestrato il modello in PyTorch, si può esportare il modello in un formato intermedio, come ONNX, e poi ottimizzarlo con TensorRT. TensorRT è uno strumento pensato per accelerare l'inferenza su GPU NVIDIA.

L'obiettivo è ridurre la latenza, cioè il tempo necessario per elaborare un singolo frame. Per raggiungere questo obiettivo, TensorRT può applicare ottimizzazioni come:

- mixed precision;
- quantizzazione;
- fusione di layer;
- ottimizzazione dei kernel;
- ottimizzazione del grafo computazionale.

La pipeline può essere:

```
modello PyTorch → esportazione ONNX → ottimizzazione TensorRT → inference engine → test FPS
```

Durante questa fase bisogna confrontare il modello originale e il modello ottimizzato. È importante verificare che l'ottimizzazione non riduca troppo la qualità della segmentazione. Il risultato ideale è un modello che mantenga buone metriche di IoU e Dice, ma che sia anche abbastanza veloce da lavorare in tempo reale.

6. Conclusione

Il progetto richiede di sviluppare una pipeline completa per la segmentazione del sangue in video laparoscopici. Il punto di partenza è HemoSet, un dataset specializzato per blood segmentation in contesto chirurgico. Il task principale è la semantic segmentation binaria, in cui ogni pixel deve essere classificato come blood o background.

La roadmap consigliata parte dall'analisi del dataset e dalla preparazione dei dati, prosegue con l'implementazione di una baseline U-Net o U-Net++, e poi evolve verso architetture più moderne come Swin-Unet, SegFormer o YOLO-based segmentation. La parte più avanzata riguarda la dual-stage localization, cioè la combinazione tra segmentazione della regione sanguigna e tracking della possibile sorgente del sanguinamento.

Infine, il progetto deve considerare il vincolo real-time. Un sistema utile in ambito chirurgico deve essere non solo accurato, ma anche veloce. Per questo è necessario misurare FPS, latenza e prestazioni del modello, eventualmente usando strumenti di ottimizzazione come TensorRT.

In sintesi, il progetto non consiste solo nell'addestrare un modello di segmentazione, ma nel costruire una pipeline completa:

```
dataset → pre-processing → modello → maschera → valutazione → tracking →  
ottimizzazione real-time
```

Il risultato atteso è un sistema capace di trasformare un frame laparoscopico in una maschera utile per identificare il sangue e supportare, in prospettiva, applicazioni di hemostasis management automatico o assistito.